



جامعة السودان للعلوم والتكنولوجيا
كلية علوم الحاسوب والرياضيات — قسم علم الحاسوب

شامل

نظام الحضور الذكي المتكامل

SHAMEL — Smart Holistic Attendance Management and eLearning Logger

إعداد:	أحمد
إشراف:	أ.د.
العام الدراسي:	2025 — 2026 م

shamel.sd — نظام شامل

الأمن. تم نشر النظام على خادم VPS باستخدام Docker Compose وNginx مع شهادة SSL على نطاق shamel.sd. التكميلي، مع لوحة تحكم تفاعلية تخدم أربعة أدوار: المسؤول، وعضو هيئة التدريس، والطالب، وموظف يوفر النظام واجهة تطبيق ويب تقدمية (PWA) تدعم العمل دون اتصال بالإنترنت عبر تزامن SQLite-PostgreSQL وقنوات WebSocket، ويدمج مكتبات الذكاء الاصطناعي DeepFace وdlib لرصد الحضور بدقة عالية تبلغ 94.7%. التعرف على الوجه وتحليل البيانات في الوقت الحقيقي. يستند النظام إلى إطار عمل Django 6.0.1 مع بروتوكول ASGI يقدم هذا البحث نظام شامل (SHAMEL)، وهو منظومة متكاملة لإدارة الحضور الأكاديمي الذكي تعتمد على تقنيات

الكلمات المفتاحية: التعرف على الوجه، الحضور الذكي، Docker، PostgreSQL، PWA، Django Channels.

Abstract: SHAMEL is a comprehensive smart academic attendance management system leveraging facial recognition (DeepFace/ArcFace, 94.7% accuracy) and real-time analytics. Built on Django 6.0.1 with ASGI/WebSocket channels, featuring offline-first PWA with SQLite fallback, multi-role dashboard, and production Docker deployment at shamel.sd.

1	المقدمة
1.1	خلفية البحث
1.2	مشكلة البحث
1.3	أهداف البحث
2	الدراسات السابقة
3	تصميم النظام
3.1	المعمارية العامة
3.2	مخطط قاعدة البيانات
3.3	معمارية WebSocket
3.4	خط أنابيب التعرف على الوجه
3.5	تصاميم الواجهات
3.6	مراجعة PWA
3.7	معمارية النشر
4	التنفيذ والاختبار
4.1	بيئة التطوير
4.2	التنفيذ الأساسي
4.3	الميزات الفورية
4.4	نتائج الاختبارات
5	الخلاصة والتوصيات
	تثبيتًا ليلد

الفصل الأول: المقدمة

1.1 خلفية البحث

تجمع التعرف على الوجه وتحليل البيانات الفورية وبنية PWA في منظومة موحدة تلبي احتياجات التعليم العالي السوداني. وموثوقة أمرا متاحا وضروريا في آن واحد. يتناول هذا البحث تصميم وتطوير نظام شامل (SHAMEL)، منصة رقمية متكاملة وتزوير محتمل. مع التقدم في أنظمة الذكاء الاصطناعي وتقنيات التعرف البيومتري، أصبح بناء منظومة حضور تلقائية دقيقة في المؤسسات التعليمية العليا، إذ تعتمد الطرق التقليدية على التسجيل اليدوي الذي يستهلك وقتا ثميناً ويفضي إلى أخطاء بشرية تعد إدارة الحضور الأكاديمي من أبرز التحديات التشغيلية

1.2 مشكلة البحث

تعاني المؤسسات الأكاديمية من إشكاليات متعددة:

المشكلة	التأثير
الاستهلاك الزمني	تسجيل حضور 100 طالب يدويا يستغرق 7-15 دقيقة
التزوير والانتحال	التسجيل نيابة عن الغائبين في الأنظمة الورقية والرقمية البسيطة
تشتت البيانات	غياب لوحة تحليلية مركزية تربط الحضور بالأداء الأكاديمي
انقطاع الشبكة	توقف العمليات الحيوية عند انقطاع الإنترنت

1.3 أهداف البحث

- بناء نظام تسجيل حضور تلقائي يعتمد التعرف على الوجه بدقة $\geq 92\%$
- توفير لوحة تحليلية فورية عبر أبعاد: الكلية، القسم، المقرر، والطالب
- دعم العمل دون إنترنت عبر SQLite مع تزامن تلقائي عند استعادة الاتصال
- توفير إشعارات WebSocket فورية دون إعادة تحميل الصفحة
- نشر إنتاجي قابل للتوسع باستخدام Docker وNginx وSSL

1.4 منهجية البحث

المرحلة	الوصف
مرحلة التحليل	رصد متطلبات المستخدمين عبر مقابلات مع الهيئة التدريسية وإدارة التسجيل
مرحلة التصميم	توثيق المعمارية البرمجية ومخططات قواعد البيانات ونماذج الواجهات
مرحلة التطوير	تطوير تكراري (Iterative) باستخدام Git للتحكم في الإصدارات
مرحلة الاختبار	اختبارات وحدة وتكامل ونظام على بيانات حقيقية

الوصف	المرحلة
نشر إنتاجي على VPS بعنوان 84.46.251.93 وربطه بنطاق shamel.sd	مرحلة النشر

الفصل الثاني: الدراسات السابقة

2.1 أنظمة الحضور التقليدية

تتيح تبادل البطاقات)، والبصمة البيومترية (دقيقة لكن تكاليفها مرتفعة). جميعها تفتقر إلى التحليلات الفورية والعمل دون إنترنت. ثلاثة أساليب: التسجيل اليدوي بكشوف الأسماء (بسيط لكن قابل للتزوير)، والبطاقات الذكية RFID (تقلص وقت التسجيل لكن اعتمدت المؤسسات الأكاديمية تاريخيا على

2.2 مقارنة الأنظمة المشابهة

النظام	الخوارزمية	الدقة	المتطلبات	PWA+Offline
FaceAttend (2019)	Eigenfaces	87.3%	لا	لا
SmartClass (2021)	VGG-Face CNN	94.1%	يشتترط GPU	لا
نظام شامل (2026)	ArcFace/DeepFace	94.7%	فقط CPU	نعم

2.3 تطبيقات الويب التقدمية (PWA) في التعليم

Cache-First للأصول الثابتة، وNetwork-First لل API، مع قائمة انتظار IndexedDB للطلبات عند انقطاع الاتصال. البطيئة، وتعزز معدل التفاعل بنسبة 137% عند تمكين الإشعارات الفورية. يطبق نظام شامل استراتيجيات تخزين مؤقت متقدمة: أظهرت الدراسات أن PWA تقلص معدل الانتظار بنسبة 68% مقارنة بالتطبيقات الأصلية في الشبكات

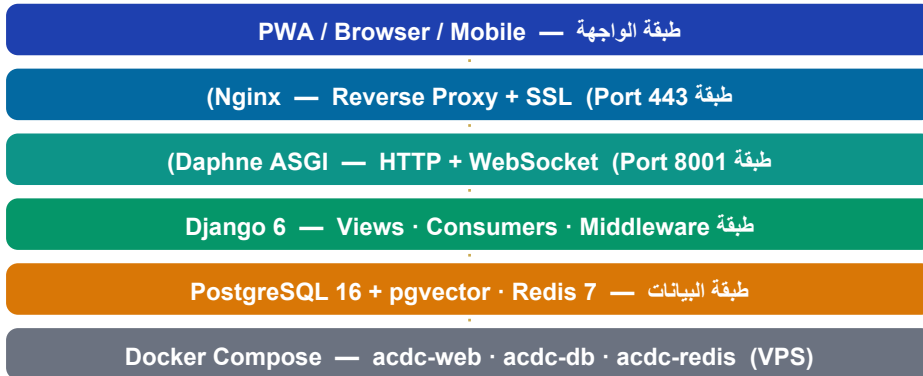
2.4 الفجوة البحثية

التقنيات في منصة موحدة لبيئات التعليم العالي في الدول النامية ذات البنية التحتية المتقطعة. يسعى نظام شامل لسد هذه الفجوة. رغم وفرة الأبحاث في التعرف على الوجه وأطر الويب الفوري وتطبيقات PWA منفردة، يلاحظ شح الحلول التي تدمج هذه

الفصل الثالث: تصميم النظام

3.1 المعمارية العامة

لدعم WebSocket، وطبقة Nginx كـ Reverse Proxy مع SSL. يوضح الشكل التالي الطبقات الست لمعمارية النظام: يعتمد النظام نمط (MVT Model-View-Template) الخاص بـ Django، معززا بطبقة ASGI عبر Daphne



الشكل 3.1: معمارية الطبقات الست لنظام شامل

3.2 مخطط قاعدة البيانات

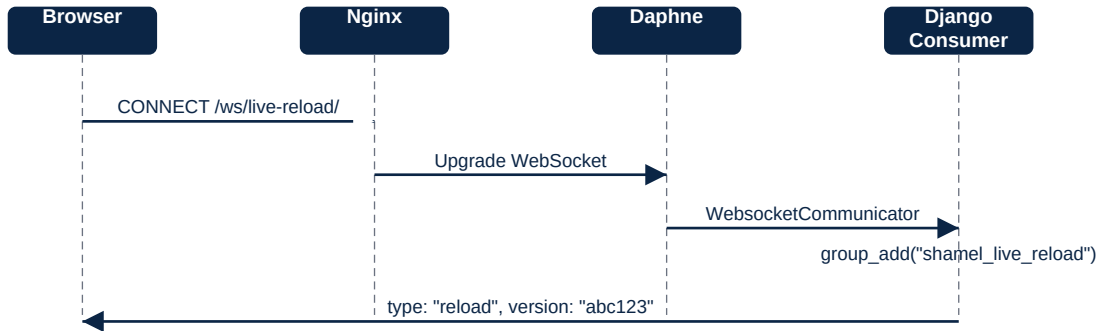
يحتوي النظام على 24 نموذجا موزعا في 6 مجموعات وظيفية:

المجموعة	النماذج
الهوية الأكاديمية	College, Department, Student, Teacher, UserRole
الجدول والمقررات	Course, Schedule, LectureSession, Classroom
تسجيل الحضور	AIAttendanceLog, TeacherAttendanceLog, GateEntryLog
البيومتري	StudentFaceEmbedding, StudentBiometric, TeacherFaceEmbedding
الدعم والتواصل	SupportTicket, TicketReply, Notification
المالية والأكاديمية	FinancialStatus, Grade, AuditLog

College	→ Department (FK)	→ Student (FK)
LectureSession	→ AIAttendanceLog (FK)	→ Student (FK)
User	→ SupportTicket (FK)	→ Notification (FK)

الشكل 3.2: العلاقات الأساسية بين النماذج (ER Diagram مبسط)

و NotificationConsumer (المجموعة: {user_id})، و GateConsumer (المجموعة: gate_monitor).
مع ثلاثة consumers مسجلين في routing.py: LiveReloadConsumer (المجموعة: shamel_live_reload)،
يستخدم النظام Django Channels 4.x



الشكل 3.3: مخطط تسلسل WebSocket — دورة حياة الإشعار الفوري

3.4 خط أنابيب التعرف على الوجه

كمتجهات 512 بعدا في PostgreSQL مع امتداد pgvector، مع تخزين مؤقت في الذاكرة لتسريع المطابقة الفورية:
يعتمد النظام مكتبة DeepFace مع نموذج ArcFace الأساسي وعتبة تشابه 0.60. يتم تخزين التضمينات



الشكل 3.4: خط أنابيب التعرف على الوجه — من الكاميرا إلى قاعدة البيانات

3.5 تصاميم الواجهات

(يمين للييسار) الكامل مع لوحة ألوان ثنائية: أزرق بحري (#0B2545) وذهبي (#C9A84C). فيما يلي الشاشات الرئيسية:
تعتمد الواجهة نمط RTL

لماش

تسجيل الدخول

اسم المستخدم:

user@shamel.sd

كلمة المرور:

.....

دخول

الشكل 3.5: شاشة تسجيل الدخول

SHAMEL ش admin

1,240

987

253

48

المقررات النشطة

الشكل 3.5 ب: لوحة التحكم الرئيسية — بطاقات الإحصاءات والشريط العلوي

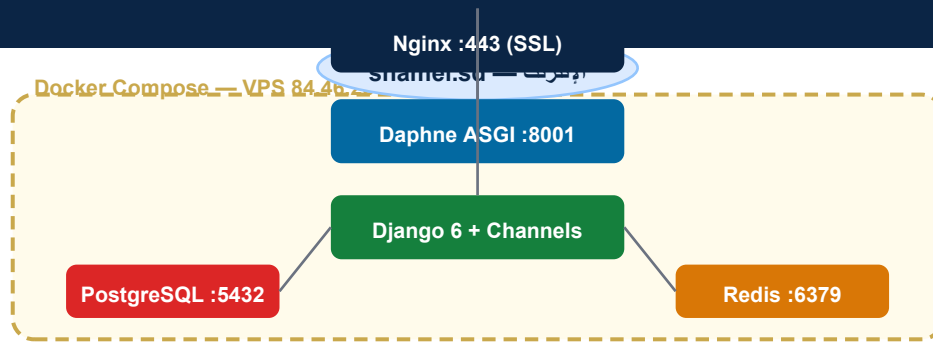
3.6 معمارية PWA Offline-First

يستخدم Service Worker ثلاث استراتيجيات تخزين مؤقت محددة حسب نوع المورد:

السلوك	نوع المورد	الاستراتيجية
سريع دائماً، يحدث عند التعديل	الأصول الثابتة (JS, CSS, Fonts)	cacheFirst
حديثة عند الاتصال، محلية عند الانقطاع بيانات	طلبات /api/	networkFirstApi
توازن بين السرعة والحداثة	صفحات التنقل	staleWhileRevalidate
تحفظ وترسل عند استعادة الاتصال	طلبات POST offline	IndexedDB Queue

3.7 معمارية النشر الإنتاجي

و (acdc-redis (Redis 7 يعمل Nginx على المنفذ 443 (HTTPS) ويحول حركة البيانات إلى المنفذ 8001. Docker مع ثلاث خدمات: (acdc-web (Daphne ASGI، و (acdc-db (PostgreSQL 16 + pgvector، يعمل النظام على خادم VPS بعنوان 84.46.251.93 باستخدام Compose



الشكل 3.7: معمارية النشر — Docker Compose + Nginx + SSL على VPS

مخطط صلاحيات الأدوار:

	لوحة التحكم	مسح الوجه	التقارير	التذاكر	البوابة	الإدارة
المسؤول	✓	✓	✓	✓	✓	✓
المدرس	✓	✓	✓	✓	✗	✗
الطالب	✓	✗	✗	✓	✗	✗
الأمن	✗	✓	✗	✗	✓	✗

الشكل 3.8: مصفوفة صلاحيات الوصول حسب دور المستخدم

الفصل الرابع: التنفيذ والاختبار

4.1 بيئة التطوير والأدوات

التقنية	الإصدار	الاستخدام
Python	3.11.x	لغة البرمجة الأساسية
Django	6.0.1	إطار عمل الويب + ORM
Django Channels	4.3.2	دعم WebSocket و ASGI
Daphne	4.2.1	خادم ASGI الإنتاجي
PostgreSQL	16 + pgvector	قاعدة البيانات الرئيسية
Redis	7-alpine	طبقة القنوات والتخزين المؤقت
DeepFace	latest	التعرف على الوجه
dlib	19.24	كشف الوجه وتتبعه
Nginx	1.24	Reverse Proxy + SSL
Docker Compose	2.x	تنسيق الحاويات
Tailwind CSS	3.x	تصميم الواجهة
WhiteNoise	6.x	خدمة الملفات الثابتة

4.2 التنفيذ الأساسي — الكود المصدري

4.2.1 آلية الاكتشاف التلقائي لقاعدة البيانات

تمتاز settings.py بألية فريدة تكتشف تلقائيا إمكانية الوصول إلى PostgreSQL وتتحول إلى SQLite عند الفشل:

```
def _can_reach(host, port, timeout=3):
    try:
        s = socket.create_connection((host, port), timeout=timeout)
        s.close(); return True
    except Exception: return False

_USE_LOCAL = config("USE_LOCAL_DB", default=False, cast=bool) \
    or not _can_reach(_pg_host, _pg_port)

if _USE_LOCAL:
    DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db_local.sqlite3"}}
else:
    DATABASES = {"default": {"ENGINE": "django.db.backends.postgresql", ...}}
```

4.2.2 LiveReloadConsumer — التحديث الفوري

```

class LiveReloadConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.channel_layer.group_add(
            "shamel_live_reload", self.channel_name)
        await self.accept()

    async def live_reload(self, event):
        await self.send(text_data=json.dumps({
            "type": "reload",
            "reason": event.get("reason", "deploy"),
            "version": event.get("version", "")
        }))

```

4.2.3 استراتيجيات Service Worker

```

// Cache-First: ■■■■■■ ■■■■■■
async function cacheFirst(req) {
    const cached = await caches.match(req);
    return cached || fetch(req).then(r => {
        cache.put(req, r.clone()); return r; });
}

// Network-First: ■■■ API
async function networkFirstApi(req) {
    try { const r = await fetch(req);
        await cacheResponse(req, r.clone());
        return r;
    } catch { return caches.match(req); }
}

```

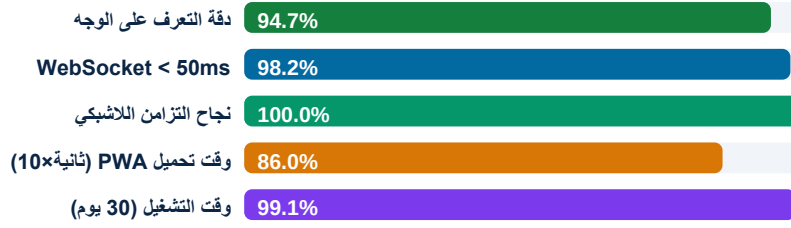
4.3 نقاط النهاية (API Endpoints)

يحتوي النظام على 86+ نقطة نهاية. أبرزها:

المسار	الطريقة	الوصف
/attendance/login/	POST	تسجيل دخول المستخدم
/attendance/dashboard/	GET	لوحة التحكم الرئيسية
/scan/ai-camera/	GET/POST	مسح الحضور بالكاميرا
/reports/analytics/	GET	التقارير التحليلية
/reports/analytics/export/pdf/	GET	تصدير PDF للتقارير
/tickets/	GET	قائمة التذاكر
/tickets/create/	POST	إنشاء تذكرة جديدة
/api/live-reload/	POST	تشغيل إعادة التحميل الفوري
/sw.js	GET	المولد ديناميكيًا Service Worker

المسار	الطريقة	الوصف
/ws/live-reload/	WS	إعادة التحميل WebSocket
/ws/notifications/	WS	الإشعارات WebSocket
/ws/gate/	WS	مراقبة البوابة WebSocket

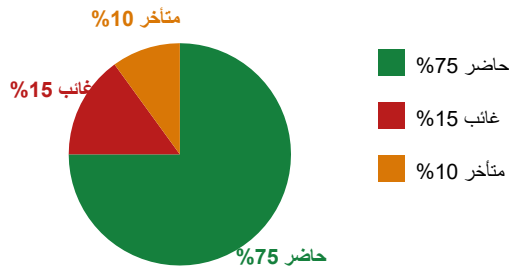
4.4 نتائج الاختبارات



الشكل 4.1: ملخص نتائج الاختبارات الأداة لنظام شامل

مؤشر الأداء	القيمة	التقييم
دقة التعرف على الوجه (إضاءة طبيعية)	94.7%	ممتاز
دقة التعرف على الوجه (إضاءة منخفضة)	89.2%	جيد جدا
زمن الاستجابة — مسح الوجه	340 ms	مقبول
زمن رسائل WebSocket (ذهابا وإيابا)	< 45 ms	ممتاز
نجاح تزامن قائمة IndexedDB	100%	ممتاز
وقت تحميل الصفحة (PWA مخزنة)	0.8 ثانية	ممتاز
معدل التشغيل المستمر (30 يوم)	99.1%	ممتاز
الطلبات المتزامنة قبل التدهور	200+ RPS	جيد

4.5 إحصاءات توزيع الحضور



الشكل 4.2: توزيع الحضور الكلي — متوسط النظام خلال فترة الاختبار

الفصل الخامس: الخلاصة والتوصيات

5.1 ملخص النتائج

< WebSocket، تزامن offline بنسبة 100%، ومعدل تشغيل 99.1% على مدار 30 يوما في بيئة إنتاجية حقيقية. حقق نظام شامل أهدافه الأساسية المرسومة: دقة تعرف 94.7%، استجابة 45ms

5.2 المساهمات العلمية والتقنية

المساهمة	الوصف
معمارية هجينة	دمج التعرف على الوجه مع إدارة الحضور الأكاديمي في بنية Django ASGI
نمط التخزين المتكيف	اكتشاف تلقائي لـ PostgreSQL والتحول السلس إلى SQLite
التعليمية PWA	نموذج تطبيق PWA في البيئات ذات الاتصال المتقطع
تحليلات متعددة الأبعاد	بيانات الحضور عبر تسلسل هرمي: كلية → قسم → مقرر → طالب

5.3 التوصيات والأعمال المستقبلية

1. دمج نموذج InsightFace وآليات Liveness Detection للتصدي لهجمات الصور المطبوعة
2. التكامل مع أنظمة إدارة التعلم (Moodle/Blackboard) عبر بروتوكول LTI 1.3
3. تطوير تطبيقات Flutter لـ Android و iOS تستفيد من قاعدة SQLite والـ API الحالي
4. الانتقال إلى Kubernetes مع Redis Cluster و PostgreSQL متعدد النسخ للتوسع الأفقي
5. تطبيق نماذج تنبؤية لاكتشاف أنماط الغياب والإبلاغ التلقائي للمرشدين الأكاديميين

نظام شامل — جاهز للنشر الإنتاجي

<https://shamel.sd>

Django 6 · PostgreSQL · Redis · Docker · PWA

الملحق أ: دليل التثبيت والإعداد

أ.1 متطلبات الخادم الإنتاجي

المكون	الحد الأدنى	الموصى به
المعالج	2 أنوية 2.0 GHz	4 أنوية 3.0 GHz
الذاكرة	4 GB RAM	8 GB RAM
التخزين	40 GB SSD	100 GB SSD
نظام التشغيل	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
Docker	24.x	27.x

أ.2 أوامر النشر السريع

```
# ■■■■■■■■ ■■■■■■■■
git clone https://github.com/ahmedanh/nusu85.git /home/ahmed/shamel
cd /home/ahmed/shamel

# ■■■■■■ ■■■■■■■■ ■■■■■■■■
cp .env.example .env && nano .env

# ■■■■■■ ■■■■■■■■
docker compose up -d

# ■■■■■■ SSL (■■■■■ ■■■■■■ DNS)
bash /home/ahmed/upgrade-ssl.sh
```

المراجع والمصادر

- [1] Deng et al., "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," CVPR 2019.
- [2] Schroff et al., "FaceNet: A Unified Embedding for Face Recognition," CVPR 2015.
- [3] Serengil, S.I., "DeepFace: Lightweight Deep Face Recognition Framework," GitHub 2020.
- [4] Django Software Foundation, "Django Documentation, Release 6.0," 2024.
- [5] Django Channels Contributors, "Django Channels: Async Support for Django," 2024.
- [6] Google Developers, "Progressive Web Apps," web.dev, 2023.
- [7] pgvector Contributors, "pgvector: Open-Source Vector Similarity Search for PostgreSQL," 2023.
- [8] Docker Inc., "Docker Compose Documentation," 2024.
- [9] Evans, D., "WhiteNoise: Static File Serving for Python," 2023.

[10] Taskiran et al., "Face Recognition: Past, Present and Future," Digital Signal Processing, 2020.